

FIAP – FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Curso de Ciência da Computação

Yago Souza Araujo – RM 575200

Katia Regina Bispo – RM 576044

João Pedro Candido Souza – RM 575884

Henrique Floriano Alcantara – RM 575651

Nicolas Klai de de França – RM 575343

RELATÓRIO OPERACIONAL DE PRÉ-DECOLAGEM

Missão Aurora Siger – Fase 1

São Paulo

2026

SUMÁRIO

1 INTRODUÇÃO	3
1.1 Organização e descrição da telemetria	3
Temperatura Interna	4
Temperatura Externa	4
Integridade Estrutural	4
Nível de Energia	5
Pressão dos Tanques	5
Status dos Módulos Críticos	5
Considerações sobre os parâmetros adotados.....	6
1.2 Algoritmo de verificação	6
1.3 Script em Python	7
1.4 Análise energética.....	11
1.4.1 Detalhamento do cálculo de análise energética.....	11
1.5 Análise assistida por IA.....	12
1.6 Reflexão crítica.....	12
1.7 Execução e resultados atualizados.....	13
1.8 Consulta individual e tabela consolidada	16
REFERÊNCIAS	17

1 INTRODUÇÃO

Este documento consolida as verificações necessárias para autorizar ou abortar o lançamento da nave Aurora Siger, bem como as decisões de engenharia, os algoritmos e as reflexões éticas e sustentáveis do projeto.

1.1 Organização e descrição da telemetria

A telemetria da nave Aurora Siger é monitorada de forma contínua, processando sinais vitais para garantir a segurança no pré-lançamento. Os parâmetros e seus limites operacionais seguros são apresentados no Quadro 1.

Quadro 1 – Parâmetros de telemetria e limites operacionais seguros

Parâmetro	Limite Operacional / Faixa Segura	Motivo de Segurança
Temperatura Interna	15°C a 35°C	Proteção de hardware e processadores.
Temperatura Externa	-100°C a 200°C	Resistência térmica estrutural do escudo.
Integridade Estrutural	Exigido 1	0 indica microfissuras ou colapso estrutural.
Nível de Energia	Mínimo de 80%	Garantia de autonomia para picos de ignição.
Pressão dos Tanques	300 kPa a 500 kPa	Vazão correta de propelente.
Status dos Módulos Críticos	Exigido 1	Subsistemas vitais 100% operacionais.

Fonte: elaborado pelos autores (2026).

Para simular o ecossistema autônomo de pré-lançamento de forma realista e automatizada, os dados de telemetria foram estruturados em um arquivo CSV (telemetry_aurora.csv). Este arquivo atua como um banco de dados sintético, contendo 10 registros (linhas) independentes.

Cada registro simula uma leitura instantânea dos sensores da nave Aurora Siger, contemplando os seis parâmetros operacionais descritos no Quadro 1. A utilização deste arquivo permite que o script em Python valide, em uma única execução, tanto cenários perfeitos de funcionamento quanto diversas situações anômalas, sem a necessidade de inserção manual de dados, testando assim a precisão e a confiabilidade do algoritmo de verificação.

Temperatura Interna

Parâmetro: 15 °C a 35 °C.

A temperatura interna representa a condição térmica no interior da nave espacial. Esse dado é importante para acompanhar o ambiente interno e verificar se os sistemas e equipamentos estão operando dentro de uma faixa térmica adequada. Em missões espaciais, sistemas de controle térmico e controle ambiental são utilizados para regular a temperatura e proteger os equipamentos e, quando aplicável, garantir condições adequadas para a tripulação.

Para a simulação da Aurora Siger, foi estabelecida a faixa de **15 °C a 35 °C** como critério de segurança. Valores fora desse intervalo são considerados anomalias pelo programa e podem indicar uma condição térmica inadequada para a operação da nave. Essa faixa foi adotada como referência para a atividade e não representa um limite universal aplicável a todas as naves espaciais.

Temperatura Externa

Parâmetro: -100 °C a 200 °C.

A temperatura externa representa a condição térmica do ambiente ao redor da nave ou de sua superfície externa. Esse parâmetro é relevante porque uma nave espacial pode enfrentar variações significativas de temperatura devido à exposição à radiação solar, à sombra e às condições de operação. O controle térmico é necessário para evitar que os componentes ultrapassem seus limites de funcionamento.

Na simulação da Aurora Siger, foi adotada a faixa de **-100 °C a 200 °C** como referência para a análise da temperatura externa. Quando o valor está fora desse intervalo, o sistema registra uma falha. Esses limites são critérios hipotéticos definidos para o projeto, pois a faixa real aceitável depende da missão, dos materiais, da órbita, do sistema de controle térmico e dos componentes monitorados.

Integridade Estrutural

Parâmetro: 0 ou 1.

A integridade estrutural representa, no modelo da atividade, uma indicação simplificada sobre a condição da estrutura da nave. A estrutura deve suportar os esforços mecânicos associados ao lançamento, à operação e às demais condições da missão. Por isso, seu acompanhamento é importante para identificar possíveis danos, deformações ou situações que comprometam a segurança do veículo.

Na Aurora Siger, o valor **1** significa que a estrutura está considerada íntegra, enquanto o valor **0** indica uma falha estrutural. Esse indicador binário foi escolhido para simplificar a tomada de decisão no programa. Em sistemas reais, a avaliação estrutural pode envolver sensores de deformação, vibração, esforços mecânicos e métodos de monitoramento da saúde estrutural, em vez de depender apenas de um valor binário.

Nível de Energia

Parâmetro: mínimo de 80%.

O nível de energia representa a quantidade percentual de energia disponível no sistema de armazenamento ou fornecimento de energia da nave. A energia é necessária para alimentar computadores, sensores, sistemas de comunicação, controle térmico e outros equipamentos essenciais à missão.

Para a simulação, foi definido que a nave deve possuir pelo menos **80% de energia disponível** para ser considerada apta à decolagem. Valores abaixo desse limite são classificados como falha pelo programa. Esse percentual é um critério hipotético da atividade, utilizado para representar uma margem de segurança energética.

Além disso, o projeto considera uma capacidade total estimada de **150 kWh** e um consumo previsto de **40 kWh**. Esses valores são premissas da simulação e permitem calcular a energia disponível e a autonomia energética estimada para cada leitura de telemetria.

Pressão dos Tanques

Parâmetro: 300 kPa a 500 kPa.

A pressão dos tanques representa a pressão interna de um reservatório da nave. Em sistemas de propulsão, os tanques e as linhas de combustível ou propelente possuem parâmetros de pressão que precisam ser monitorados para verificar as condições de funcionamento do sistema. A pressão também pode ser importante para garantir que os fluidos sejam conduzidos adequadamente durante a operação.

Na Aurora Siger, foi estabelecida a faixa de **300 kPa a 500 kPa** como condição aceitável para a simulação. Valores abaixo ou acima dessa faixa são considerados falhas pelo programa. Esses limites são específicos do modelo proposto e não devem ser interpretados como valores reais universais para tanques de naves espaciais, pois a pressão adequada depende do tipo de tanque, do fluido armazenado, do sistema de propulsão e das especificações de engenharia.

Status dos Módulos Críticos

Parâmetro: 0 ou 1.

O status dos módulos críticos indica, no modelo da atividade, se os módulos essenciais da nave estão funcionando adequadamente. Esses módulos podem representar sistemas responsáveis por funções importantes, como comunicação, controle, energia, navegação ou gerenciamento da missão.

Na simulação, o valor **1** representa módulos em condição operacional normal, enquanto o valor **0** indica que existe uma falha em algum módulo crítico. O programa utiliza esse indicador para decidir se a nave pode prosseguir com a decolagem.

Em sistemas espaciais reais, o acompanhamento do estado dos subsistemas faz parte dos dados de telemetria de engenharia, também conhecidos como dados de monitoramento da saúde da

nave ou *housekeeping telemetry*. Esses dados podem incluir estados de equipamentos, medições elétricas, temperaturas, pressões e outras informações operacionais.

Considerações sobre os parâmetros adotados

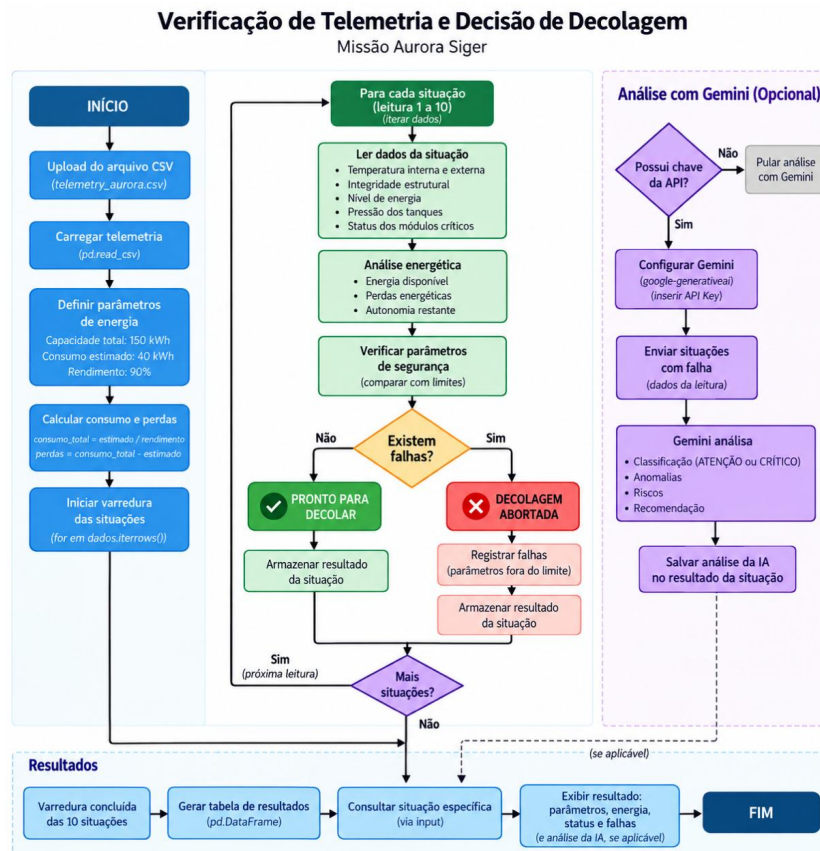
Os campos utilizados no arquivo CSV foram escolhidos para representar diferentes aspectos da saúde operacional da nave Aurora Siger: condições térmicas, integridade estrutural, disponibilidade de energia, pressão dos tanques e estado dos módulos críticos.

A escolha desses parâmetros foi baseada em conceitos utilizados no monitoramento de sistemas espaciais. Entretanto, os valores-limite empregados na simulação — temperaturas, pressão, nível mínimo de energia e indicadores binários — são **premissas definidas para o desenvolvimento da atividade da FIAP**. Em uma missão real, esses limites seriam determinados a partir de requisitos de engenharia, características dos componentes, testes, análises de segurança e especificações da missão.

1.2 Algoritmo de verificação

O algoritmo de verificação atua por meio de desvios condicionais que avaliam os parâmetros capturados contra seus limites seguros de forma independente. Se todos os parâmetros estiverem dentro da faixa, o status final emitido é “PRONTO PARA DECOLAR”. Caso contrário, o lançamento é imediatamente classificado como “DECOLAGEM ABORTADA”, e as falhas simultâneas são acumuladas para diagnóstico. A representação visual e a lógica passo a passo dessa arquitetura são apresentadas na Figura 1.

Figura 1 – Fluxograma de verificação de telemetria e decisão de decolagem



Fonte: elaborado pelos autores (2026).

1.3 Script em Python

O ecossistema embarcado da nave foi simulado computacionalmente por meio de um script na linguagem Python. Para garantir precisão de processamento autônomo, utilizou-se a biblioteca Pandas na extração otimizada dos dados a partir de um dataset em formato .csv, evitando preenchimentos manuais irreais. O script percorre os registros de telemetria aplicando as regras do algoritmo de verificação e exibe os diagnósticos no terminal em tempo real.

A implementação atualizada (arquivo Aurora.ipynb) mantém a mesma lógica apresentada no fluxograma. O código percorre cada registro do arquivo CSV, calcula a energia disponível e a autonomia restante, verifica todos os parâmetros de segurança, registra as falhas encontradas e armazena o resultado de cada situação para consulta posterior, conforme o Quadro 2.

Quadro 2 – Trecho do algoritmo de verificação e armazenamento dos resultados

```
# Imports do projeto
import pandas as pd
import time
from google.colab import files
```

```
# Upload do arquivo CSV
arquivo_enviado = files.upload()

print('Carregando telemetria da nave Aurora Siger...\n')

dados = pd.read_csv("telemetry_aurora.csv")

# Dados de energia
capacidade_total = 150          # kWh
consumo_estimado = 40            # kWh
rendimento = 0.90               # 90%

# Consumo total considerando o rendimento
consumo_total = consumo_estimado / rendimento

# Perdas energéticas
perdas_energia = consumo_total - consumo_estimado

print('Arquivo carregado com sucesso!')
print(f'Total de registros: {len(dados)}')

# Lista para armazenar os resultados
resultados = []

# Varredura e validação dos parâmetros de segurança
for indice, linha in dados.iterrows():

    print(f'--- Analisando Leitura #{indice + 1} ---')

    temperatura_interna = linha['internal_temperature']
    temperatura_externa = linha['external_temperature']
    integridade_estrutural = linha['structural_integrity']
    nivel_energia = linha['energy_level']
    pressao_tanques = linha['tank_pressure']
    status_modulos = linha['module_status']

    # Análise energética
    energia_disponivel = (capacidade_total * nivel_energia) / 100
    autonomia_restante = energia_disponivel - consumo_total

    print(f'Energia Disponível: {energia_disponivel:.2f} kWh')
    print(f'Consumo estimado (com rendimento): {consumo_total:.2f} kWh')
    print(f'Perdas energéticas: {perdas_energia:.2f} kWh')
    print(f'Autonomia restante estimada: {autonomia_restante:.2f} kWh\n')

    decolagem_segura = True
    falhas = []

    # Parâmetros seguros da missão
    parametros = {
        "Temperatura Interna": 15 <= temperatura_interna <= 35,
        "Temperatura Externa": -100 <= temperatura_externa <= 200,
        "Integridade Estrutural": integridade_estrutural == 1,
        "Nível de Energia": nivel_energia >= 80,
        "Pressão dos Tanques": 300 <= pressao_tanques <= 500,
        "Status dos Módulos Críticos": status_modulos == 1
    }

    # Verificação dos parâmetros
    for nome_parametro, status_valido in parametros.items():
        if not status_valido:
            decolagem_segura = False
            falhas.append(nome_parametro)
```

```

# Decisão final
if decolagem_segura:
    status = "PRONTO PARA DECOLAR"
    print('STATUS: PRONTO PARA DECOLAR\n')
else:
    status = "DECOLAGEM ABORTADA"
    print('STATUS: DECOLAGEM ABORTADA.\n')
    print('Os Sistemas abaixo falharam:\n')

    for motivo_falha in falhas:
        print(f' - {motivo_falha}')

    print()

# Armazena o resultado da leitura
resultado = {
    "leitura": indice + 1,
    "internal_temperature": temperatura_interna,
    "external_temperature": temperatura_externa,
    "structural_integrity": integridade_estrutural,
    "energy_level": nivel_energia,
    "tank_pressure": pressao_tanques,
    "module_status": status_modulos,
    "available_energy": energia_disponivel,
    "remaining_energy": autonomia_restante,
    "safe_takeoff": decolagem_segura,
    "status": status,
    "failures": falhas
}

resultados.append(resultado)

print('-' * 70)
print()

print(f'Varredura completa dos {len(dados)} registros finalizada com sucesso.')

# Consulta individual de uma leitura
numero_leitura = int(input("Digite o número da situação que deseja consultar: "))

resultado = resultados[numero_leitura - 1]

print(f'\nLeitura: {resultado["leitura"]}')
print(f'Temperatura interna: {resultado["internal_temperature"]} °C')
print(f'Temperatura externa: {resultado["external_temperature"]} °C')
print(f'Nível de energia: {resultado["energy_level"]}%')
print(f'Pressão dos tanques: {resultado["tank_pressure"]} kPa')
print(f'Energia disponível: {resultado["available_energy"]:.2f} kWh')
print(f'Autonomia restante: {resultado["remaining_energy"]:.2f} kWh')
print(f>Status: {resultado["status"]}')
print(f>Falhas: {resultado["failures"]}')

# Criação da tabela de resultados
tabela_resultados = pd.DataFrame(resultados)
tabela_resultados

# Instalação e configuração da API do Gemini
!pip install -q google-genai

from google import genai
from getpass import getpass

CHAVE_GEMINI = getpass("Cole sua Gemini API Key: ")
cliente = genai.Client(api_key=CHAVE_GEMINI)

def analisar_com_gemini(dados_falha):

```

```

    mensagem = f"""
Você é o sistema analítico de suporte à missão da nave Aurora Siger.
Analise esta leitura de telemetria que apresentou falha no pré-lançamento:

- Temperatura Interna: {dados_falha['internal_temperature']} °C (Ideal: 15 a 35)
- Temperatura Externa: {dados_falha['external_temperature']} °C (Ideal: -100 a 200)
- Integridade Estrutural: {dados_falha['structural_integrity']} (Exigido: 1)
- Nível de Energia: {dados_falha['energy_level']}% (Mínimo: 80)
- Pressão dos Tanques: {dados_falha['tank_pressure']} kPa (Ideal: 300 a 500)
- Status dos Módulos Críticos: {dados_falha['module_status']} (Exigido: 1)

Forneça uma resposta objetiva estruturada exatamente nos seguintes tópicos:
CLASSIFICAÇÃO: (ATENÇÃO ou CRÍTICO)
ANOMALIAS: (Descreva o que causou o problema)
RISCOS: (Impacto estrutural ou operacional)
RECOMENDAÇÃO: (Ajuste corretivo de engenharia)
"""

    resposta = cliente.models.generate_content(
        model="gemini-3.5-flash-lite",
        contents=mensagem
    )

    return resposta.text

print("Gemini configurado com sucesso!")

# Análise das falhas com o Gemini
for resultado in resultados:

    if resultado["safe_takeoff"] == False:

        print(f'--- Gemini analisando Leitura #{resultado["leitura"]} ---\n')

        dados_falha = {
            'internal_temperature': resultado['internal_temperature'],
            'external_temperature': resultado['external_temperature'],
            'structural_integrity': resultado['structural_integrity'],
            'energy_level': resultado['energy_level'],
            'tank_pressure': resultado['tank_pressure'],
            'module_status': resultado['module_status']
        }

        try:
            print('[IA] Acionando triagem assistida do Gemini...\n')

            analise_ia = analisar_com_gemini(dados_falha)
            resultado["analise_ia"] = analise_ia

            print(analise_ia)

        except Exception as erro:
            print(f'Aviso de conexão com a API: {erro}')

        print('\n' + '=' * 100 + '\n')
        time.sleep(1)

# Exibição da análise do Gemini
numero_leitura = 3

resultado = resultados[numero_leitura - 1]

if "analise_ia" in resultado:
    print(resultado["analise_ia"])
else:

```

```
print("Esta leitura ainda não possui uma análise do Gemini salva.")

# Atualização da tabela final
tabela_resultados = pd.DataFrame(resultados)
tabela_resultados
```

Fonte: elaborado pelos autores (2026), com base no notebook Aurora.ipynb.

GitHub: https://github.com/YagoSDev/Aurora_Siger

1.4 Análise energética

A gestão termodinâmica é crucial para a missão. Para validar a autonomia energética, foi considerado um banco de baterias com capacidade hipotética de 150 kWh. O pico útil exigido na decolagem foi estimado em 40 kWh, e a eficiência do sistema em 90%. Assim, o cálculo modelado encontrou o consumo total bruto de 44,44 kWh, sendo os 4,44 kWh adicionais referentes a perdas térmicas e dissipativas. Em cada iteração, o sistema converte a porcentagem atual da bateria em kWh disponível, abate o consumo e verifica matematicamente a energia restante para pós-decolagem.

1.4.1 Detalhamento do cálculo de análise energética

O cálculo foi dividido em duas fases: a definição das constantes físicas do motor/bateria e a matemática dinâmica aplicada a cada linha de telemetria.

Definição das constantes técnicas (passo fixo). Antes de processar os dados, o sistema define as características do banco de baterias e do consumo de decolagem:

- Capacidade Total (capacidade_total): 150 kWh — representa um banco de baterias de íon-lítio de alta densidade, projetado para sustentar computadores de bordo, telemetria de alta frequência e sistemas inerciais durante o pré-lançamento.
- Consumo Útil Estimado (consumo_estimado): 40 kWh — é o pico instantâneo de energia exigido para acionar os ignitores, pressurizar tanques e fechar válvulas principais no momento do empuxo.
- Eficiência do Sistema (rendimento): 0,90 (90%).

Cálculo de perdas e consumo real (termodinâmica). Com base na Primeira Lei da Termodinâmica e no Efeito Joule, sabe-se que os circuitos dissipam calor. O código calcula essas perdas antes da decolagem:

- Consumo Total Real: calculado pela divisão do consumo útil pela eficiência da nave ($40 / 0,90$). O resultado é 44,44 kWh.
- Perdas Energéticas: subtraindo o consumo útil do consumo total real ($44,44 - 40$), obtêm-se as perdas. O resultado é 4,44 kWh, que representa o calor dissipado de forma irreversível nos condutores e conversores.

Cálculo dinâmico da autonomia (a cada leitura do CSV). Dentro do laço de repetição, o sistema utiliza a porcentagem atual da bateria (`nivel_energia`) e aplica a fórmula final:

- Energia Disponível = $(\text{capacidade_total} \times \text{nivel_energia}) / 100$.
- Autonomia Restante = Energia Disponível – Consumo Total Real.

1.5 Análise assistida por IA

Como um diferencial na arquitetura, o software conta com a integração da API do Google Gemini (modelo `gemini-3.5-flash-lite`). A inteligência artificial é invocada de forma programática toda vez que a decolagem é abortada, recebendo o conjunto anômalo de dados. O modelo atua no suporte, analisando e devolvendo, em tempo de execução, um laudo estruturado contendo a classificação de risco (Atenção/Crítico), as causas das anomalias, os riscos operacionais e a recomendação técnica direcionada à equipe de engenharia.

Na versão atual do notebook, toda a configuração e execução do Google Gemini foi concentrada no final do código. Dessa forma, a análise principal pode ser executada integralmente sem chave de API. O Gemini é acionado apenas para leituras que já receberam o status `DECOLAGEM ABORTADA`, e sua resposta é armazenada como análise complementar.

1.6 Reflexão crítica

Ética e responsabilidade. Em sistemas críticos, a automação deve ser explicável, confiável e auditável. O Machine Learning e a Inteligência Artificial podem auxiliar na identificação de anomalias e na análise dos dados, mas as regras de segurança e a decisão final devem permanecer sob responsabilidade humana e institucional. Dessa forma, evita-se que uma previsão incorreta da IA coloque a missão, os equipamentos ou pessoas em risco.

Impacto social. A exploração espacial pode trazer benefícios importantes para a sociedade, contribuindo para avanços científicos, comunicação, navegação, previsão do tempo e monitoramento ambiental. Entretanto, essas atividades também exigem grandes investimentos financeiros, tecnológicos e energéticos. Por isso, é importante avaliar se os benefícios gerados justificam os recursos utilizados e como essas tecnologias podem contribuir para a sociedade.

Impacto ambiental e sobre os animais. Lançamentos espaciais podem causar impactos no meio ambiente, como emissão de gases, poluição sonora e geração de resíduos. O ruído intenso e as alterações provocadas nas regiões próximas às bases de lançamento também podem afetar animais, especialmente aves e outras espécies sensíveis ao som e às mudanças em seu habitat. Além disso, componentes descartados durante missões podem atingir oceanos ou outras áreas naturais. Por esse motivo, locais e métodos de lançamento devem ser planejados buscando reduzir danos à fauna, à flora e aos ecossistemas.

Sustentabilidade. A análise das perdas energéticas realizada no projeto contribui para identificar desperdícios e melhorar a eficiência do sistema. Em aplicações reais, a sustentabilidade também envolve reduzir o consumo de recursos, aumentar a vida útil dos equipamentos, utilizar fontes de energia mais limpas, reaproveitar componentes quando possível e realizar a destinação adequada de resíduos eletrônicos e materiais utilizados nas missões.

Uso responsável de IA. Neste projeto, a IA atua como uma ferramenta de apoio e não possui autoridade para liberar uma decolagem. As verificações determinísticas continuam sendo responsáveis por identificar condições que violem os limites de segurança. Essa separação é importante porque modelos de IA podem apresentar erros ou classificações imprecisas. Assim, a tecnologia complementa a análise sem substituir critérios objetivos de segurança e a supervisão humana.

1.7 Execução e resultados atualizados

A versão atual do programa analisa dez situações de telemetria. Em cada leitura são apresentados a energia disponível, o consumo considerando o rendimento, as perdas energéticas, a autonomia restante, o status final e, quando necessário, os parâmetros que causaram o aborto da decolagem.

Figura 2 – Varredura das situações 1 a 3

```

--- Analisando Leitura #1 ---
... Energia Disponível: 138.75 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 94.31 kWh

STATUS: PRONTO PARA DECOLAR

-----

--- Analisando Leitura #2 ---
Energia Disponível: 127.50 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 83.06 kWh

STATUS: PRONTO PARA DECOLAR

-----

--- Analisando Leitura #3 ---
Energia Disponível: 135.00 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 90.56 kWh

STATUS: DECOLAGEM ABORTADA.

Os Sistemas abaixo falharam:

- Temperatura Interna

```

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

As situações 1 e 2 atenderam a todos os limites programados e foram classificadas como PRONTO PARA DECOLAR. A situação 3 foi abortada por temperatura interna fora da faixa.

Figura 3 – Varredura das situações 4 a 6

```

--- Analisando Leitura #4 ---
... Energia Disponível: 142.50 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 98.06 kWh

STATUS: DECOLAGEM ABORTADA.

Os Sistemas abaixo falharam:

- Temperatura Externa
- Integridade Estrutural

-----

--- Analisando Leitura #5 ---
Energia Disponível: 111.75 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 67.31 kWh

STATUS: DECOLAGEM ABORTADA.

Os Sistemas abaixo falharam:

- Nível de Energia

-----

--- Analisando Leitura #6 ---
Energia Disponível: 132.00 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 87.56 kWh

STATUS: DECOLAGEM ABORTADA.

Os Sistemas abaixo falharam:

- Pressão dos Tanques
- Status dos Módulos Críticos

```

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

A situação 4 apresentou temperatura externa inadequada e falha de integridade estrutural. A situação 5 foi abortada por nível de energia inferior ao mínimo de 80%. A situação 6 apresentou pressão dos tanques fora da faixa e falha nos módulos críticos.

Figura 4 – Varredura das situações 7 a 9

```
--- Analisando Leitura #7 ---
... Energia Disponível: 148.50 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 104.06 kWh

STATUS: PRONTO PARA DECOLAR

-----

--- Analisando Leitura #8 ---
Energia Disponível: 120.00 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 75.56 kWh

STATUS: PRONTO PARA DECOLAR

-----

--- Analisando Leitura #9 ---
Energia Disponível: 121.80 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 77.36 kWh

STATUS: PRONTO PARA DECOLAR

-----
```

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

As situações 7, 8 e 9 permaneceram dentro dos limites definidos e receberam autorização para decolagem.

Figura 5 – Varredura da situação 10 e finalização da análise

```
--- Analisando Leitura #10 ---
Energia Disponível: 75.00 kWh
Consumo estimado (com rendimento): 44.44 kWh
Perdas energéticas: 4.44 kWh
Autonomia restante estimada: 30.56 kWh

STATUS: DECOLAGEM ABORTADA.

Os Sistemas abaixo falharam:

- Temperatura Interna
- Temperatura Externa
- Integridade Estrutural
- Nível de Energia
- Pressão dos Tanques
- Status dos Módulos Críticos

-----

Varredura completa dos 10 registros finalizada com sucesso.
```

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

A situação 10 concentrou falhas em todos os parâmetros principais e, por isso, foi classificada como DECOLAGEM ABORTADA.

1.8 Consulta individual e tabela consolidada

A consulta individual passou a solicitar o número da situação por meio de input. Assim, não é necessário alterar manualmente o código para visualizar outra leitura. O número informado é usado para acessar o resultado correspondente já armazenado na lista, conforme ilustra a Figura 6.

Figura 6 – Consulta de uma situação pelo número informado pelo usuário

```
Digite o número da situação que deseja consultar: 1

Leitura: 1
Temperatura interna: 25.0 °C
Temperatura externa: 45.0 °C
Nível de energia: 92.5%
Pressão dos tanques: 350.0 kPa
Energia disponível: 138.75 kWh
Autonomia restante: 94.31 kWh
Status: PRONTO PARA DECOLAR
Falhas: []
```

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

Ao final da execução, a lista de resultados é convertida em um DataFrame do Pandas. A tabela reúne as dez leituras e permite comparar os valores de telemetria, energia, autonomia, status de decolagem e falhas registradas (Figura 7).

Figura 7 – Tabela consolidada com os resultados das dez situações

	leitura	internal_temperature	external_temperature	structural_integrity	energy_level	tank_pressure	module_status	available_energy	remaining_energy	safe_takeoff	status	failures
0	1	25.0	45.0	1.0	92.5	350.0	1.0	138.75	94.305556	True	PRONTO PARA DECOLAR	[]
1	2	18.5	-10.2	1.0	85.0	410.5	1.0	127.50	83.055556	True	PRONTO PARA DECOLAR	[]
2	3	48.2	150.0	1.0	90.0	380.0	1.0	135.00	90.555556	False	DECOLAGEM ABORTADA	[Temperatura Interna]
3	4	22.0	-120.5	0.0	95.0	320.0	1.0	142.50	98.055556	False	DECOLAGEM ABORTADA	[Temperatura Externa, Integridade Estrutural]
4	5	29.4	30.0	1.0	74.5	450.0	1.0	111.75	67.305556	False	DECOLAGEM ABORTADA	[Nível de Energia]
5	6	31.0	85.0	1.0	88.0	250.0	0.0	132.00	87.555556	False	DECOLAGEM ABORTADA	[Pressão dos Tanques, Status dos Módulos Crít...
6	7	20.0	10.0	1.0	99.0	300.0	1.0	148.50	104.055556	True	PRONTO PARA DECOLAR	[]
7	8	34.9	199.9	1.0	80.0	499.0	1.0	120.00	75.555556	True	PRONTO PARA DECOLAR	[]
8	9	15.1	-99.8	1.0	81.2	301.2	1.0	121.80	77.355556	True	PRONTO PARA DECOLAR	[]
9	10	40.0	210.0	0.0	50.0	150.0	0.0	75.00	30.555556	False	DECOLAGEM ABORTADA	[Temperatura Interna, Temperatura Externa, Int...

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

Quando a etapa opcional da API é utilizada, o Gemini analisa apenas as leituras que tiveram a decolagem abortada, e a coluna `analise_ia` passa a ser exibida na tabela consolidada (Figura 8).

Figura 8 – Tabela consolidada com a análise complementar do Gemini

	leitura	internal_temperature	external_temperature	structural_integrity	energy_level	tank_pressure	module_status	available_energy	remaining_energy	safe_takeoff	status	failures	analise_fa
0	1	25.0	45.0	1.0	92.5	350.0	1.0	138.75	94.305556	True	PRONTO PARA DECOLAR	[]	NaN
1	2	18.5	-10.2	1.0	85.0	410.5	1.0	127.50	83.055556	True	PRONTO PARA DECOLAR	[]	NaN
2	3	48.2	150.0	1.0	90.0	380.0	1.0	135.00	90.555556	False	DECOLAGEM ABORTADA	[Temperatura Interna]	CLASSIFICAÇÃO: ATENÇÃOvAnOMALIAS: Temperatu...
3	4	22.0	-120.5	0.0	95.0	320.0	1.0	142.50	98.055556	False	DECOLAGEM ABORTADA	[Temperatura Externa, Integridade Estrutural]	CLASSIFICAÇÃO: CRÍTICOvAnOMALIAS: Integrida...
4	5	29.4	30.0	1.0	74.5	450.0	1.0	111.75	87.305556	False	DECOLAGEM ABORTADA	[Nível de Energia]	CLASSIFICAÇÃO: CRÍTICOvAnOMALIAS: Nível de...
5	6	31.0	85.0	1.0	88.0	250.0	0.0	132.00	87.555556	False	DECOLAGEM ABORTADA	[Pressão dos Tanques, Status dos Módulos Crít...	CLASSIFICAÇÃO: CRÍTICOvAnOMALIAS: A falta n...
6	7	20.0	10.0	1.0	99.0	300.0	1.0	148.50	104.055556	True	PRONTO PARA DECOLAR	[]	NaN
7	8	34.9	199.9	1.0	80.0	499.0	1.0	120.00	75.555556	True	PRONTO PARA DECOLAR	[]	NaN
8	9	15.1	-99.8	1.0	81.2	301.2	1.0	121.80	77.355556	True	PRONTO PARA DECOLAR	[]	NaN
9	10	40.0	210.0	0.0	90.0	150.0	0.0	75.00	30.555556	False	DECOLAGEM ABORTADA	[Temperatura Interna, Temperatura Externa, Int...	CLASSIFICAÇÃO: CRÍTICOvAnOMALIAS: v-tempe...

Fonte: captura de tela da execução do notebook Aurora.ipynb (2026).

REFERÊNCIAS

GOOGLE. Gemini API. Google AI for Developers. Disponível em: <https://ai.google.dev/gemini-api/docs?hl=pt-br>.

GOOGLE. API keys. Google AI Studio. Disponível em: <https://aistudio.google.com/api-keys>.

INSTITUTO DE AERONÁUTICA E ESPAÇO. Projetos Espaciais e Veículos Lançadores. Disponível em: dcta.mil.br.

INSTITUTO DE ESTUDOS AVANÇADOS. Divisão de Aerotermodinâmica e Hipersonônica. Disponível em: cta.br.

INSTITUTO TECNOLÓGICO DE AERONÁUTICA. Divisão de Engenharia Aeroespacial. Disponível em: <http://www.ita.br/aer/aesp>.

NASA. Armstrong Flight Research Center. Autonomous Systems. Disponível em: <https://nasa.gov>.

EUROPEAN SPACE AGENCY. ESA: European Space Agency. Disponível em: <https://esa.int>.

FIAP – FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA. Materiais de apoio da disciplina de Ciência da Computação – Fase 1. São Paulo: FIAP, 2026.

O código-fonte, o notebook, o arquivo CSV, os registros de execução e os demais materiais do projeto estão disponíveis no repositório do **GitHub**: https://github.com/YagoSDev/Aurora_Siger